

Task Scheduling

May 15, 2026

Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Outline

- 1 Linux scheduling framework
- 2 Scheduling Classes
- 3 Data structures
- 4 Running Scheduler
- 5 System Wide Operations
- 6 Tasks Specific Operations

1 Linux scheduling framework

2 Scheduling Classes

3 Data structures

4 Running Scheduler

5 System Wide Operations

6 Tasks Specific Operations

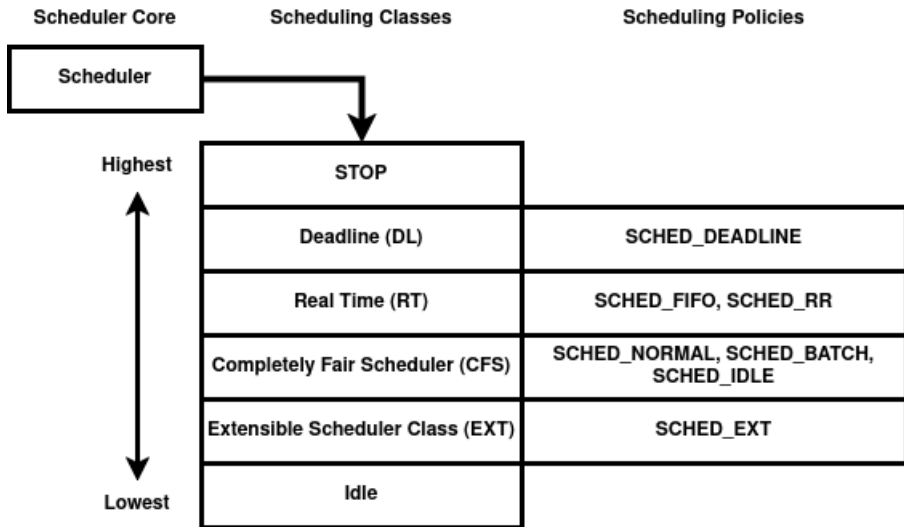
Overview (I)

- The **scheduler** (or dispatcher) is the part of the kernel responsible, at run-time, **for allocating CPUs to tasks for execution** and the **scheduling classes are responsible for selecting those tasks**.
- The scheduling classes encapsulate scheduling policies.
- **These scheduling classes are hierarchically organized by priority** and the scheduler inquires each scheduling class in a decreasing priority order for a ready task.
- Linux has six main scheduling classes:
 - STOP
 - Deadline (DL)
 - Real-Time (RT)
 - Completely Fair Scheduling (CFS)
 - Extensible Scheduler Class (EXT)
 - Idle

Overview (II)

- 1 The scheduler core first inquires the STOP scheduling class for a ready task.
- 2 If STOP scheduling class does not have any ready task, then it inquires the DL scheduling class.
- 3 If DL does not have any ready task, it proceeds by inquiring RT.
- 4 If RT does not have any ready task it proceeds inquiring CFS.
- 5 If CFS does not have ready task it proceeds inquiring EXT.
- 6 If EXT does not have ready task then it reaches the idle scheduling class, used for the idle task.
- 7 Every processor has an **idle task** in its ready-queue that is executed whenever there is no other ready task.

Overview (III)



- 1 Linux scheduling framework
- 2 Scheduling Classes**
- 3 Data structures
- 4 Running Scheduler
- 5 System Wide Operations
- 6 Tasks Specific Operations

STOP

- Only available for symmetric multi-processing (SMP) (`stop_machine()` is not used in uni-processor).
- It can **preempt everything and is preempted by nothing**.
- **Mechanism to stop running everything else and run a specific function on CPU(s)**.
- No scheduling policies.
- There is one kernel thread per CPU: `migration/N`.
- It is used by task migration, CPU hotplug, RCU, ftrace, clockevents, etc.

DL

- **SCHED_DEADLINE Policy: A real-time policy based on the Earliest Deadline First (EDF) algorithm.**
- **EDF Algorithm:** The task with the closest deadline is always chosen to run next.
- **Timing Constraints:** Tasks specify three parameters: runtime (worst-case execution time), period (activation interval), and deadline (time to complete from activation).
- **Constant Bandwidth Server (CBS):** This mechanism enforces the timing constraints, ensuring no task exceeds its allocated runtime and protecting system stability.
- **Use Cases:** Ideal for complex real-time systems where deadline guarantees are critical, such as video streaming or autonomous vehicle control.

RT

- **Two scheduling policies:**
 - **SCHED_FIFO (First-In, First-Out):** A static-priority policy where a task runs until it blocks, yields, or is preempted by a higher-priority task.
 - It has no time slices.
 - **SCHED_RR (Round-Robin):** Similar to SCHED_FIFO but uses a fixed time slice (100ms time slice by default).
 - If a task exhausts its slice, it moves to the back of the queue for its priority level.
- **Priority Range:** Real-time priorities range from 1 (lowest) to 99 (highest) and are always higher than normal task priorities.
- **Use Cases:** Essential for applications with strict timing constraints, like industrial control systems, robotics, and high-frequency trading.
- **Starvation Risk:** A high-priority real-time task **can potentially starve all lower-priority tasks if it never blocks or yields.**

CFS

- Default Scheduling class: **CFS is the scheduling class for all non-real-time processes:**
 - `SCHED_NORMAL`: Normal tasks (`SCHED_OTHER`, the older name for `SCHED_NORMAL`)
 - `SCHED_BATCH`: Batch (non-interactive) tasks
 - `SCHED_IDLE`: Low priority tasks
- **Core Principle:** It aims for "ideal multitasking" by giving each task a fair proportion of the processor's execution time.
- **Virtual Runtime (`vruntime`):** CFS tracks the amount of time a process has run, normalized by its priority and the process with the lowest `vruntime` runs next.
 - Priority is used to set task's weight, that affects `vruntime`.
 - Higher the weight, slower will `vruntime` increase.
 - Task's priority is calculated by $120 + \text{nice value}$ (-20 to +19).

EEVDF: Earliest Eligible Virtual Deadline First

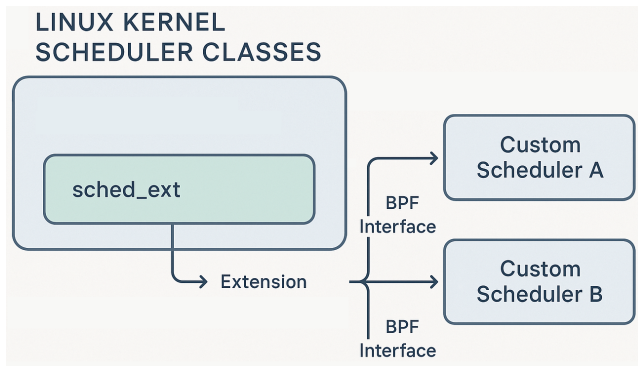
- The CFS Core Problem
 - It answers **who is most owed?** but not **who needs CPU soonest?** or **has this task exceeded its share?**
- EEVDF adds virtual deadlines to the fairness model (used by CFS).
- CFS asks: **who has used the least CPU?**
 - CFS picks the smallest vruntime.
- EEVDF asks: **who has the most urgent deadline, among those who are eligible to run?**
 - EEVDF adds an eligibility filter (lag) and picks the earliest deadline among eligible tasks.

EEVDF Core Concepts

- EEVDF extends the vruntime model with two additional per-task values:
 - (as CFS)Virtual Runtime (vruntime): Accumulated “fair” CPU time.
 - Higher weight = slower accumulation.
 - Lag = fair_vruntime - actual_vruntime
 - Positive: task is owed CPU time (eligible)
 - Negative: task exceeded fair share (not eligible)
 - Virtual Deadline: When task’s current time slice “expires” in virtual time.
 - Computed from eligibility time and slice/weight.
- EEVDF Selection Rule
 - Pick task with earliest virtual deadline among eligible tasks ($\text{lag} \geq 0$)

EXT

- EXT is a scheduling class **whose behavior can be defined by a set of BPF programs** – the BPF scheduler.
- EXT exports a full scheduling interface so that any scheduling algorithm can be implemented on top.



Idle

- **Lowest priority scheduling class.**
- No scheduling policies.
- One kernel thread (idle) per CPU: `swapper/N`.
- Idle thread runs only when nothing else is runnable on a CPU.
- Idle thread may take the CPU to low power state.

- 1 Linux scheduling framework
- 2 Scheduling Classes
- 3 Data structures**
- 4 Running Scheduler
- 5 System Wide Operations
- 6 Tasks Specific Operations

Data structures (I)

- For **every active processes in the system**, Linux kernel create an instance of `struct task_struct` to manage them.
 - The kernel must know, for instance, the process's priority, whether it is running on a CPU or blocked on an event, what address space has been assigned to it, which files it is allowed to address, and so on.
 - `struct task_struct` is composed by data fields for all this issues
- Each **CPU has its own runqueue**, as an instance of `struct rq`
 - A `struct rq` is a container for all processes in a `TASK_RUNNING` state.
 - Further, it contains many other fields for managing the CPU.

Data structures (II)

- The **Linux scheduler is modular, enabling different algorithms/policies to schedule different types of tasks.**
- An algorithm's implementation is wrapped in a so called **scheduling class.**
- A **scheduling class offers an interface to the main scheduler skeleton** which it can use **to handle tasks according to the implemented algorithm(s).**
- A scheduling class is an instance of `struct sched_class` data structure.

struct task_struct

- /include/linux/sched.h

```
struct task_struct {  
    ...  
    struct sched_entity se;  
    struct sched_rt_entity rt;  
    struct sched_dl_entity dl;  
    struct sched_dl_entity *dl_server;  
#ifdef CONFIG_SCHED_CLASS_EXT  
    struct sched_ext_entity scx;  
#endif  
    const struct sched_class *sched_class;  
    ...  
};
```

- This data structure incorporates data structures per scheduling classes: struct sched_entity, struct sched_rt_entity and struct sched_dl_entity for CFS, RT and DL scheduling classes, respectively.
- sched_class pointer specifies the scheduling class used to manage scheduling operations of the task.

Scheduling Entities

- A scheduling entity **is an abstract representation of a process, thread, or task within the kernel that is subject to scheduling.**
 - In simpler terms, it is a data structure that **holds all the information necessary for the kernel to manage and track how a particular process or thread gets CPU time.**
- `/include/linux/sched.h`

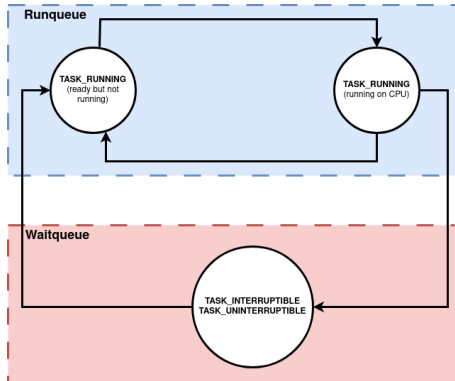
```
struct sched_entity {
    ...
    struct rb_node    run_node;
    u64    deadline;
    ...
    u64    vruntime;
    ...
}
```

```
struct sched_dl_entity {
    struct rb_node    rb_node;
    /* Maximum runtime for each instance */
    u64    dl_runtime;
    /* Relative deadline of each instance */
    u64    dl_deadline;
    /* Separation of two instances (period) */
    u64    dl_period;
    ...
}
```

```
struct sched_rt_entity {
    struct list_head    run_list;
    ...
    unsigned int    time_slice;
    ...
}
```

Runqueue

- In the Linux kernel, **the runqueue (represented by struct `rq`) is the core per-CPU data structure that tracks all processes currently in a runnable state.**
 - It acts as a **container of runnable processes**



struct rq (I)

- /kernel/sched/sched.h

```

struct rq {
    raw_spinlock_t    __lock;
    /* Per class runqueue modification mask; bits in class order. */
    unsigned int      queue_mask;
    unsigned int      nr_running;
    ...
    struct cfs_rq cfs;
    struct rt_rq rt;
    struct dl_rq dl;
    ...
    struct sched_dl_entity fair_server;
    ...
#ifdef CONFIG_SCHED_PROXY_EXEC
    struct task_struct __rcu *donor; /* Scheduling context */
    struct task_struct __rcu *curr; /* Execution context */
#else
    union {
        struct task_struct __rcu *donor; /* Scheduler context */
        struct task_struct __rcu *curr; /* Execution context */
    };
#endif
    ...
    struct task_struct *idle;
    struct task_struct *stop;

```

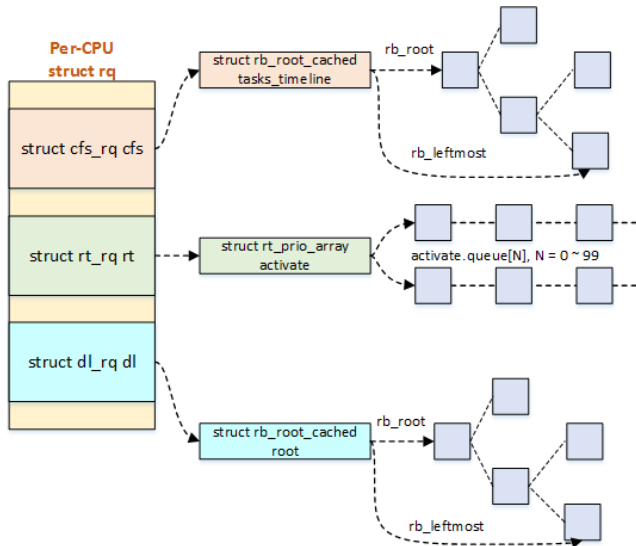
`struct rq (II)`

- It **keeps track of all runnable tasks assigned to a particular CPU**:
 - A lock to synchronize scheduling operations for this CPU
 - The `queue_mask` acts as a bitmask where each bit corresponds to a specific scheduling class (such as STOP, DL, RT, CFS or EXT).
 - Its purpose is to allow the scheduler to quickly determine which classes have tasks waiting to be run without having to scan the individual runqueues for every class.
 - Actually, **runqueue incorporates sub-runqueues per scheduling classes**:
 - `struct dl_rq`, `struct cfs_rq` and `struct rt_rq`, for DL, CFS and RT scheduling classes, respectively.
 - `fair_server` it is the entry point to a mechanism designed to solve the CFS tasks starvation.
 - Actually, it is a "fake" DL task that ensures CFS tasks are not completely starved of CPU time by runaway RT tasks.

struct rq (III)

- CONFIG_SCHED_PROXY_EXEC is a kernel configuration option that enables Proxy Execution.
 - Proxy Execution **is a generalized form of priority inheritance designed for the Linux kernel scheduler to solve priority inversion problems**, particularly across different scheduling classes.
 - Its purpose is to allow the scheduler to quickly determine which classes have tasks waiting to be run without having to scan the individual runqueues for every class.
- Pointer(s) to the currently running (`curr` or `donor`).
- Pointers to the stop (`stop`) and the idle (`idle`) tasks.

struct rq (IV)



struct cfs_rq

```

/* CFS-related fields in a runqueue */
struct cfs_rq {
    struct load_weight load;
    unsigned int    nr_queued;
    unsigned int    h_nr_queued; /* SCHED_{NORMAL,BATCH,IDLE} */
    unsigned int    h_nr_runnable; /* SCHED_{NORMAL,BATCH,IDLE} */
    unsigned int    h_nr_idle; /* SCHED_IDLE */
    ...
    struct rb_root_cached tasks_timeline;
    /*
     * 'curr' points to currently running entity on this cfs_rq.
     * It is set to NULL otherwise (i.e when none are currently running).
     */
    struct sched_entity *curr;
    struct sched_entity *next;
    ...
};

```

- `tasks_timeline` is the red-black tree root used to manage and rank runnable tasks based on their virtual runtime (`vruntime`).

struct rt_rq (I)

```

struct rt_rq {
    struct rt_prio_array active;
    unsigned int   rt_nr_running;
    unsigned int   rr_nr_running;
    struct {
        int curr; /* highest queued rt task prio */
        int next; /* next highest */
    } highest_prio;
    ...
};

```

```

/*
 * This is the priority-queue data structure of the RT scheduling class:
 */
struct rt_prio_array {
    DECLARE_BITMAP(bitmap, MAX_RT_PRIO+1); /* include 1 bit for delimiter */
    struct list_head queue[MAX_RT_PRIO];
};

```

struct rt_rq (II)

- In the RT scheduling class, **tasks are assigned a fixed priority from 0 to 99**.
- Instead of looping through all 100 possible priority levels to see which ones have runnable tasks, the scheduler consults the `active.bitmap`.
- Each bit in this 100-bit mask represents one priority level:
 - Bit = 1: There is at least one task waiting at this priority.
 - Bit = 0: The priority level is empty.
- If the RT bitmap is non-zero, the scheduler knows instantly that it must handle RT tasks before even looking at the Fair (CFS) tasks, further streamlining the decision path.
- Modern CPUs have specialized instructions (like Bit Scan Forward – `bsf` – or Count Leading Zeros – `clz`) designed to find the "first set bit" in a word.

struct dl_rq (l)

```

/* Deadline class' related fields in a runqueue */
struct dl_rq {
    /* runqueue is an rbtree, ordered by deadline */
    struct rb_root_cached root;
    unsigned int    dl_nr_running;
    /*
     * Deadline values of the currently executing and the
     * earliest ready task on this rq. Caching these facilitates
     * the decision whether or not a ready but not running task
     * should migrate somewhere else.
     */
    struct {
        u64 curr;
        u64 next;
    } earliest_dl;
    ...
};

```

- The Deadline (DL) scheduling class in Linux implements a **Global Earliest Deadline First (G-EDF) queue by combining local per-CPU runqueues** with a coordinated "push and pull" migration mechanism across a root domain.

`struct dl_rq` (II)

- Actually, **there is no single monolithic "global" queue in memory** – to avoid the massive lock contention that would arise – the scheduler achieves global behavior using distributed structures:
 - **Per-CPU Red-Black Trees:** Each CPU maintains its own `struct dl_rq`, where tasks are ordered by their absolute deadlines.
 - **Root Domains:** Multi-processor systems are partitioned into "root domains" (usually representing all CPUs or specific set of CPUs – a cluster).
 - Within a root domain (managed using the `struct root_domain` data structure), the scheduler ensures that the earliest deadline tasks run on the available CPUs.

struct dl_rq (III)

- Global EDF properties are maintained through two active migration operations:
 - **Push Operations:** Triggered when a new task becomes ready on a local CPU.
 - If that task has an earlier deadline than one running on a different CPU in the root domain, the scheduler "pushes" it to the most suitable CPU (one running a task with a later deadline or an idle one).
 - **Pull Operations:** Triggered when a CPU is about to run a lower-priority task (or becomes idle).
 - It scans other runqueues in the root domain to "pull" the task with the earliest deadline that isn't currently running.

struct sched_class (I)

- /kernel/sched/sched.h

```

struct sched_class {
...
    /*
    * idle: 0
    * ext: 1
    * fair: 2
    * rt: 4
    * dl: 8
    * stop: 16
    */
    unsigned int queue_mask;

    void (*enqueue_task) (struct rq *rq, struct task_struct *p, int flags);
    bool (*dequeue_task) (struct rq *rq, struct task_struct *p, int flags);
    void (*wakeup_preempt) (struct rq *rq, struct task_struct *p, int flags);
    struct task_struct *(*pick_task) (struct rq *rq, struct rq_flags *rf);
    struct task_struct *(*pick_next_task) (struct rq *rq, struct task_struct *prev,
    struct rq_flags *rf);
    void (*put_prev_task) (struct rq *rq, struct task_struct *p, struct task_struct
        *next);
    void (*set_next_task) (struct rq *rq, struct task_struct *p, bool first);
    ...
    void (*task_tick) (struct rq *rq, struct task_struct *p, int queued);
};

```

struct sched_class (II)

- `queue_mask`: it is a bitmask where each bit represents a scheduling class and
 - If a bit is 1, that class has runnable tasks.
 - If a bit is 0, the scheduler skips that entire class immediately.
- `enqueue_task`: Called when **a task enters a runnable state**.
- `dequeue_task`: Called when **a task is no longer runnable**.
- `wakeup_preempt`: This function checks if **a task that entered the runnable state should preempt the currently running task**.
- `pick_next_task`: This function **chooses the most appropriate task eligible to run next**.
- `task_tick`: This function is mostly called from time tick functions;

struct sched_class (III)

- All existing scheduling classes in the kernel **are in an array which is ordered by the priority of the scheduling class.**
 - The highest priority is in the first position and the lowest in the last position.

```
#define SCHED_DATA    \
STRUCT_ALIGN();    \
__sched_class_highest = .; \
*(__stop_sched_class) \
*(__dl_sched_class) \
*(__rt_sched_class) \
*(__fair_sched_class) \
*(__idle_sched_class) \
__sched_class_lowest = .;
```

- The scheduling class array iteration begins always in the first position of the array.

```
#define for_class_range(class, _from, _to) \
    for (class = (_from); class < (_to); class++)

#define for_each_class(class) \
    for_class_range(class, __sched_class_highest, __sched_class_lowest)
```

struct thread_info (I)

- /arch/x86/include/asm/thread_info.h

```

struct thread_info {
    unsigned long  flags; /* low level flags */
    unsigned long  syscall_work; /* SYSCALL_WORK_ flags */
    u32    status; /* thread synchronous flags */
    #ifdef CONFIG_SMP
    u32    cpu; /* current CPU */
    #endif
};

```

- It is **architecture-specific data structure** used for low-level task data that must be accessed frequently or quickly by assembly code.
- It **holds hardware-specific flags, status bits, and pointers** essential for context switching.
- It is embedded into `struct task_struct` data structure

```

struct task_struct {
    struct thread_info thread_info;
    unsigned int    __state;
    ...
}

```

`struct thread_info` (II)

- In the scheduling context, **the unsigned long flags plays a relevant role.**
- It is a bitmask of various flag values (all in the style `TIF_<FOO>`, where `TIF` is an abbreviation for **thread_info flag** (`/include/asm-generic/thread_info_tif.h`).
 - Whenever `TIF_NEED_RESCHED` flag is set, the currently running task should be preempted (or at least a schedule event should happen).
- `resched_curr` function **marks the currently executing task to be preempted.**
 - This sets the `TIF_NEED_RESCHED` flag in the task structure.

- 1 Linux scheduling framework
- 2 Scheduling Classes
- 3 Data structures
- 4 Running Scheduler**
- 5 System Wide Operations
- 6 Tasks Specific Operations

The Scheduler Framework

- Tasks (processes and threads) are managed through **a modular framework where the core scheduler delegates the actual scheduling logic to different scheduling classes.**
 - The core scheduler code does not need to know the details of every scheduling algorithm (implement into a scheduling class).
 - It simply calls generic hooks like `enqueue_task` or `pick_next_task`.
- The scheduler core performs broadly two type of operations:
 - System wide operations
 - Load Balancing
 - Select the next task to execute
 - ...
 - These operations have as a start point the array of scheduling classes
 - Task specific operations
 - Enqueue and dequeue to/from runqueue
 - ...
 - This operations have as a start point the task (via `sched_class` pointer)

The scheduler code (I)

- A subtle yet key misconception regarding how scheduling works is unfortunately held by many: **we imagine that some kind of kernel thread (or some such entity) called the “scheduler” is present, which periodically runs and schedules tasks.**
 - **This is just plain wrong;**
- In Linux, **scheduling is carried out by the process contexts themselves, when in kernel mode.**
- In fact, **the scheduling code is always run by the process context that is currently executing the code of the kernel.**

The scheduler code (II)

- The scheduler **must never be invoked from an atomic context**—which includes interrupt context or while holding a spinlock.
 - Atomic context **refers to situations where the kernel cannot sleep, block, or relinquish the CPU** because it is handling a critical task that must complete without interruption.
- `TIF_NEED_RESCHED` is a flag used to signal **rescheduling request**.
 - **Instead of interrupting a task immediately at any random point, the kernel sets this flag and then checks for it at specific, safe "preemption points".**
- This mechanism is divided into two parts:
 - When is the `TIF_NEED_RESCHED` bit set?
 - When is the `TIF_NEED_RESCHED` bit checked?

Setting `TIF_NEED_RESCHED` bit

- **Timer Ticks (`sched_tick`):**
 - Every time a hardware timer interrupt fires (the "tick"), the kernel could call `resched_curr` function to set the flag.
- **Task Wakeups (`wake_up_process`):**
 - When a sleeping task becomes "runnable" (e.g., its I/O finished), the scheduler checks if this new task has a higher priority than the one currently running.
 - If it does, it calls `resched_curr` function to preempt the current task.
- **New Task Creation (`wake_up_new_task`):**
 - When a process is forked, the kernel may set the flag if the new child task is deemed "more deserving" of the CPU than the parent.
- **Priority and/or Scheduling Class Changes:**
 - If a task's priority and/or the scheduling class is/are changed (e.g., via `nice` or `sched_setscheduler`), the kernel may set the flag to immediately reflect the new context.

Checking `TIF_NEED_RESCHED` bit

- **Return from System Call**

- When a task is about to leave kernel mode and return to userspace, the kernel checks for pending work, including this flag.

- **Return from Interrupt (IRQ)**

- After any hardware interrupt (like a keyboard press or disk event) is handled, the kernel checks if a higher-priority task was woken up or if the current task's time is up.

- **Enabling Preemption**

- In general, any switch from non-preemptible to preemptible mode within the kernel is an opportunity point (when `preempt_enable` function is called) to check the flag.

Calling the Scheduler

- The invocation of `schedule` function is:
 - Performed in process context only, and
 - only at certain specific **scheduling points** that are sprinkled throughout the kernel code paths.
- Any explicit (or implicit) call to `schedule` function (e.g. when a blocking call's issued)
- Obviously, an explicit or implicit invocation of `schedule` function while in the process context will trigger it being executed.

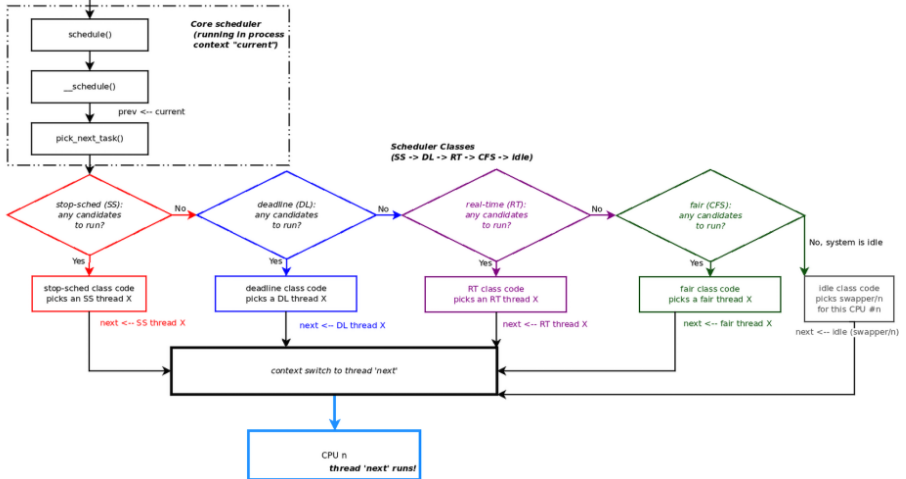
- 1 Linux scheduling framework
- 2 Scheduling Classes
- 3 Data structures
- 4 Running Scheduler
- 5 System Wide Operations**
- 6 Tasks Specific Operations

The Scheduler Framework (I)

- When scheduler core function is invoked, **it iterates over each of the scheduling classes in a predefined priority order, asking each one if it has a task that is ready to execute.**
- Eventually, if no other scheduling class has task to execute, the idle task is always ready to execute.
 - The idle task is the "fallback" of the Linux scheduler, designed to ensure the CPU always has something to execute.
- The context switch performs the switching of the CPU from one task to another.

System Wide Operations

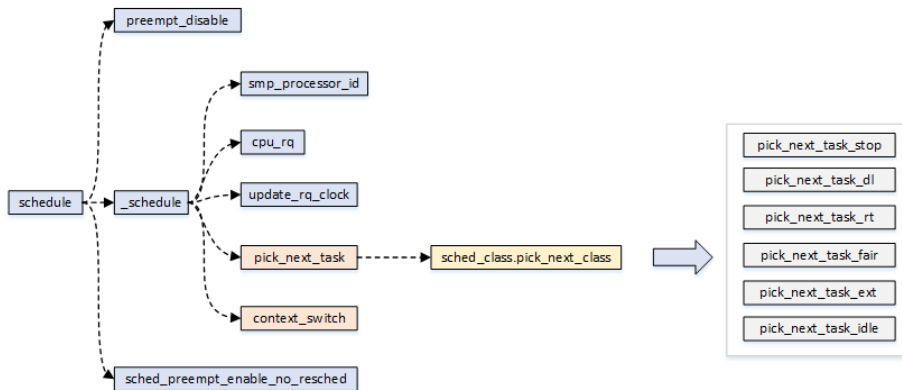
For this processor (CPU #n; n = 0, 1, 2...):



The Scheduler Framework (III)

- `schedule()` - "Public" (exported symbol) API called by the rest of the kernel to trigger rescheduling.
- `__schedule()`: The internal "heart" of the scheduler
- `pick_next_task()`: Scans scheduling classes to find the best candidate for the CPU.
- `context_switch()`: The low-level architecture-specific code that swaps tasks on CPU.

The Scheduler Framework (IV)



The Scheduler Entry Point

```
static __always_inline void __schedule_loop(int sched_mode)
{
    do {
        preempt_disable();
        __schedule(sched_mode);
        sched_preempt_enable_no_resched();
    } while (need_resched());
}

asmlinkage __visible void __sched schedule(void)
{
    struct task_struct *tsk = current;

#ifdef CONFIG_RT_MUTEXES
    lockdep_assert(!tsk->sched_rt_mutex);
#endif

    if (!task_is_running(tsk))
        sched_submit_work(tsk);
    __schedule_loop(SM_NONE);
    sched_update_worker(tsk);
}

EXPORT_SYMBOL(schedule);
```

`__schedule` function (I)

- The `__schedule` is the "heart" of the scheduler framework
- This is the function that the rest of the kernel uses to invoke the process scheduler, deciding which process to run and then running it.
- Its main goal is to find the next task to be run.
- It has two `struct task_struct` pointers, `prev` and `next` that are set with the currently executing task (which will relinquish CPU) and the next task to be executed (which will be assigned to CPU), respectively.

__schedule function (II)

```

static void __sched notrace __schedule(int sched_mode){
    struct task_struct *prev, *next;
    ...
    unsigned long prev_state;
    struct rq_flags rf;
    struct rq *rq;
    int cpu;
    ...
    cpu = smp_processor_id();
    rq = cpu_rq(cpu);
    prev = rq->curr;
    ...
    local_irq_disable();
    ...
    rq_lock(rq, &rf);
    ...
    try_to_block_task(rq, prev, &prev_state, !task_is_blocked(prev));
    ...
    next = pick_next_task(rq, rq->donor, &rf);
    is_switch = prev != next;
    if (likely(is_switch)) {
        ...
        rq = context_switch(rq, prev, next, &rf);
    }
    ...
}

```

__schedule function: main points (I)

- Since the Linux kernel is pre-emptive, it can happen that a task executing code in kernel space is involuntarily pre-empted by a higher priority task.
- ❶ It assigns the current executing task to `prev` pointer (`prev = rq->curr`)
- ❷ It disables the interrupts by calling `local_irq_disable`.
- ❸ It locks the current CPU's runqueue (`rq_lock`)
- ❹ It examines the state of the currently executing task, `prev`.
 - If it is not runnable, then it should be removed from the runqueue.
 - To remove a task from the runqueue, `try_to_block_task` is called which internally calls the `dequeue_task` hook of the task's scheduling class.
 - If it is runnable, then it is left in the runqueue.

`__schedule` function: main points (II)

- 5 Next, it is time to pick the next task to be assigned to the CPU calling `pick_next_task` function.
- 6 After that, it checks if `pick_next_task` found a new task or if it picked the same task again that was running before.
 - If the latter is the case, no task switch is performed and the current task just keeps running.
 - It calls `__pick_next_task` function
- 7 If a new task is found, which is the more likely case, the actual task switch is executed by calling `context_switch`.
 - Internally, `context_switch` switches to the new task's memory map and swaps register state and stack.
 - It also unlocks the `rq`

__pick_next_task function (I)

```

/*
 * Pick up the highest-prio task:
 */
static inline struct task_struct *
__pick_next_task(struct rq *rq, struct task_struct *prev, struct rq_flags *rf)
{
    const struct sched_class *class;
    struct task_struct *p;
    ...
    for_each_active_class(class) {
        if (class->pick_next_task) {
            p = class->pick_next_task(rq, prev, rf);
            ...
            if (p)
                return p;
        } else {
            p = class->pick_task(rq, rf);
            ...
            if (p) {
                put_prev_set_next_task(rq, prev, p);
                return p;
            }
        }
    }
    BUG(); /* The idle class should always have a runnable task. */
}

```

`__pick_next_task` function (I)

- The `__pick_next_task` function is the core decision-making routine in the Linux kernel scheduler.
- It is called by the `__schedule()` function to determine which task should run next on a given CPU.
- Its goal is to identify the highest-priority runnable task currently available.

- 1 Linux scheduling framework
- 2 Scheduling Classes
- 3 Data structures
- 4 Running Scheduler
- 5 System Wide Operations
- 6 Tasks Specific Operations**

Scheduling Class (I)

- The Linux kernel uses a `struct sched_class` variable for each scheduling class and such variable is created using `DEFINE_SCHED_CLASS(name)` macro (which create the `<name>_sched_class` variable, for instance `idle_sched_class`).

```
/*  
 * Simple, special scheduling class for the per-CPU idle tasks:  
 */  
DEFINE_SCHED_CLASS(idle) = {  
    .queue_mask = 0,  
    /* no enqueue/yield_task for idle tasks */  
    /* dequeue is not valid, we print a debug message there: */  
    .dequeue_task = dequeue_task_idle,  
    .wakeup_preempt = wakeup_preempt_idle,  
    ...  
};
```

Scheduling Class (II)

- The `struct sched_class` acts as an interface containing function pointers that the core scheduler calls during specific events:
 - `enqueue_task(...)`: Called when a task becomes runnable and is added to the class's runqueue.
 - `dequeue_task(...)`: Called when a task is no longer runnable (e.g., it blocks or finishes).
 - `pick_next_task(...)`: The most critical hook; the core scheduler calls this to ask the class for the best task to run right now.
 - `task_tick(...)`: Called by the system timer to allow the class to update accounting data (like `vruntime`) and decide if the current task should be preempted.

Tasks (I)

- Every task in the kernel is represented by a `struct task_struct`.
- `*sched_class` points to the specific scheduling class the task belongs to.
- This structure contains also a `sched_entity` (or other) member, which stores the scheduling-specific data.

```
struct task_struct {  
    ...  
    struct sched_entity se;  
    ...  
    const struct sched_class *sched_class;  
    ...  
}
```

```
struct sched_entity {  
    ...  
    struct rb_node run_node;  
    ...  
    u64 exec_start;  
    u64 sum_exec_runtime;  
    u64 vruntime;  
    ...  
}
```

Tasks (II)

- By default, when a process calls `fork` function, the child task inherits the scheduling policy, priority, and the corresponding `sched_class` pointer from its parent.
- However, a task can change the scheduling policy using the `sched_setscheduler` system call.

sched_setscheduler (I)

```
int sched_setscheduler(struct task_struct *p, int policy, const struct sched_param
    *param){
    return __sched_setscheduler(p, policy, param, true);
}
```

```
static int __sched_setscheduler(struct task_struct *p, int policy, const struct
    sched_param *param, bool check){
    ...
    return __sched_setscheduler(p, &attr, check, true);
}
```

```
int __sched_setscheduler(struct task_struct *p, const struct sched_attr *attr, bool
    user, bool pi){
    ...
    prev_class = p->sched_class;
    next_class = __setscheduler_class(policy, newprio);
    ...
    p->sched_class = next_class;
    p->prio = newprio;
    ...
}
```

sched_setscheduler (II)

```
const struct sched_class *__setscheduler_class(int policy, int prio)
{
    if (dl_prio(prio))
        return &dl_sched_class;

    if (rt_prio(prio))
        return &rt_sched_class;

#ifdef CONFIG_SCHED_CLASS_EXT
    if (task_should_scx(policy))
        return &ext_sched_class;
#endif

    return &fair_sched_class;
}
```

Task Specific Scheduling Operations

- When the core scheduler needs to perform an action, it uses the `task_struct` to find the correct `sched_class` functions

Task Wake Up (I)

- Defined in `kernel/sched/core.c`

```
int wake_up_process(struct task_struct *p){  
    return try_to_wake_up(p, TASK_NORMAL, 0);  
}
```

```
int try_to_wake_up(struct task_struct *p, unsigned int state, int wake_flags){  
    ...  
    ttwu_queue(p, cpu, wake_flags);  
    ...  
    return success;  
}
```

```
static void ttwu_queue(struct task_struct *p, int cpu, int wake_flags){  
    ...  
    ttwu_do_activate(rq, p, wake_flags, &rf);  
    ...  
}
```

Task Wake Up (II)

```
static void ttwu_do_activate(struct rq *rq, struct task_struct *p, int wake_flags,
                           struct rq_flags *rf){
    ...
    activate_task(rq, p, en_flags);
    wakeup_preempt(rq, p, wake_flags);
    ...
}
```

```
void activate_task(struct rq *rq, struct task_struct *p, int flags){
    ...
    enqueue_task(rq, p, flags);
}
```

```
void enqueue_task(struct rq *rq, struct task_struct *p, int flags){
    ...
    p->sched_class->enqueue_task(rq, p, flags);
    ...
}
```

Task Wake Up (III)

```
void wakeup_preempt(struct rq *rq, struct task_struct *p, int flags)
{
    struct task_struct *donor = rq->donor;

    if (p->sched_class == donor->sched_class)
        donor->sched_class->wakeup_preempt(rq, p, flags);
    else if (sched_class_above(p->sched_class, donor->sched_class))
        resched_curr(rq);
    ...
}
```

`sched_tick` function (I)

- The function `sched_tick` is called regularly by a timer interrupt, called **tick**.
 - This function **gets called by the timer code**, with `HZ` frequency.
- Its purpose is to update runqueue clock, CPU load and runtime counters of the currently running task.
- It calls the scheduling class hook `task_tick` of the currently executing task `task` update for the corresponding class.
 - At this point it can mark the current executing task to be preempted, by calling `resched_curr` function.
- Load balancing is invoked if SMP is configured.

sched_tick function (II)

- Defined in kernel/sched/core.c

```
void sched_tick(void)
{
    int cpu = smp_processor_id();
    struct rq *rq = cpu_rq(cpu);
    /* accounting goes to the donor task */
    struct task_struct *donor;
    ...

    if (dynamic_preempt_lazy() && tif_test_bit(TIF_NEED_RESCHED_LAZY))
        resched_curr(rq);

    donor->sched_class->task_tick(rq, donor, 0);

    ...
}
```

sched_tick function (III)

